

Argo MAC

Garbling with Elliptic Curve MACs

Liam Eagen* Ying Tong Lai †
{ ideal }

January 18, 2026

Off-chain cryptography enables more expressive smart contracts for Bitcoin [1]. Recent work, including BitVM [2, 3, 4] and Glock [5], use SNARKs to prove arbitrary computation, and garbled circuits to verifiably move proof verification off-chain. In this note, we define a new garbling primitive, Argo MAC, that enables over $1000\times$ more efficient garbled SNARK verifiers. Argo MAC efficiently translates from an encoding of the bit decomposition of a curve point to a homomorphic MAC of that point. These homomorphic MACs enable much more efficient garbling. In subsequent work, we will describe how to use Argo MAC to construct garbled SNARK verifiers for pairing-based SNARKs.

1 Introduction

Garbling schemes [6] are a powerful technique that underlie many higher-level cryptographic protocols like Secure Function Evaluation (SFE), Conditional Disclosure of Secrets (CDS), and Multiparty Computation (MPC). A garbling scheme is a two-party protocol between a Garbler and an Evaluator. In the first phase, the Garbler “garbles” a function and sends the garbling to the Evaluator. In the second phase, the Garbler shares some encoding of an input with the Evaluator, which allows them to compute the function on that, and only that, particular input.

Many of the earliest, and most practical, garbling schemes rely on some variant of Yao’s protocol [7]. This protocol takes a circuit, consisting of a DAG of gates connected by wires. Each gate takes several wires as input and produces some number of output wires. In the original scheme, the wires are binary and each gate encodes a binary logic gate. Subsequent work has extended this notion to a larger class of circuits [8, 9, 10, 11].

To garble the circuit, the Garbler assigns each wire a Message Authentication Code (MAC) key or randomized encoding scheme. They “garble” the value of that wire by encoding it under this scheme. The Garbler assigns to each gate some additional cryptographic information such that, given input wire encodings, the Evaluator can compute the encoding of the output wires. Crucially, this is the only way the Evaluator can compute valid encodings of the output wires.

In Yao’s scheme, each gate is encoded as an encrypted truth table. The input wire MACs for each row are used as the key to encrypt the output wire. Starting with the free XOR technique [12], some garbling schemes [13, 14, 15, 16, 17, 18] exploited the fact that for certain types of gates there exist encoding schemes that allow the Evaluator to evaluate the gate with no, or very little, additional information. This reduces the size of the garbled circuit and the complexity of evaluation.

For XOR, this family of techniques exploits “homomorphic MACs” (HMACs) that are linear with respect to the XOR operation. Other techniques, like information theoretic partial garbling (ITPG) [11], exploit a similar trick over the additive group of general finite fields. More recent work has even demonstrated the existence of fully homomorphic MACs which allow evaluating both addition and multiplication in $\mathbb{F}_2$, yielding constant-sized garbling tables [16, 17, 18].

*liameagen@protonmail.com

†yingtong.lai@gmail.com

1.1 BitVM and Glock

A recent line of work has shown how to use garbling schemes to augment Bitcoin with SNARK verification. Bitcoin is not capable of directly verifying complex programs, which limits its ability to build complex smart contracts, in particular SNARK verifiers. However, Bitcoin can verify the correct encoding of wires in a garbled circuit for certain encoding schemes. For example, in Yao’s garbled circuits, each input bit can have one of two possible encodings, and Bitcoin can verify that each bit is a valid encoding by checking hash pre-images. More sophisticated techniques [5, 19] are able to verify encodings of wires over larger, but still small, alphabets (e.g. byte values). We call such encodings that can be decomposed as strings over a small alphabet “projective.”

We can couple this functionality with an off-chain garbled circuit to reveal a secret conditional on a predicate. During setup, the Garbler constructs a circuit for a predicate $\mathcal{C}$, and during Evaluation reveals an encoding of an input x . The validity of this encoding can be verified by Bitcoin. The Evaluator uses this encoding to derive an encoding of the output of $\mathcal{C}(x) = y$. Bitcoin can also verify the correctness of the encoding of y .

Suppose x is a SNARK proof, and $\mathcal{C}$ is the SNARK verification circuit. If the SNARK is invalid, then $y = 0$. In an interactive BitVM-style protocol (e.g. Glock), the Evaluator can present the encoding of $y = 0$ as a “fraud proof” that x is not valid. This can be used to condition the release of funds on SNARK validity, which enables a kind of “optimistic” [20] SNARK verification on Bitcoin. Many projects are using variants of this technique to build sophisticated protocols on top of Bitcoin, which were previously believed to be impossible.

1.2 Rich Wire Encodings

The SNARK used by most Glock-like protocols is Groth16 [21], chosen for its small proof size. Some also use variants of other elliptic curve based SNARKs like Pari [22]. These SNARK verifier circuits are all naturally expressed as elliptic curve pairing¹ equations. These can in turn be expressed as simple formulae or circuits, where wire values take on elliptic curve points or other elements of large groups.

We might hope that there is some way to combine the free XOR technique with this circuit and encode curve points in a way that respects curve operations. This might allow us to very efficiently garble SNARK verifiers. However, we need this encoding to be projective with respect to a small alphabet in order to use it with Bitcoin. To encode elliptic curve points in a projective way, we decompose the curve point into some string over a small alphabet. Unfortunately, this destroys the structure of the curve point as an element of an algebraic group.

Note that this is not the case for all groups. For example, the additive group of a field admits a simple projective encoding that respects the additive structure: bit decomposition. Since bit recomposition is $\mathbb{F}$ -linear, everything just works. Unfortunately, this is likely impossible for elliptic curves. If we could find a homomorphic, projective encoding of points then we could mount an index calculus attack on the curve group, which is believed to be hard and is the basis for their unique efficiency compared to other discrete log-hard groups.

1.3 Argo MAC

Argo MAC is a technique to bridge this gap: that is, to efficiently transform a projective encoding of an elliptic curve point into a HMAC over an elliptic curve group. Of course, this is possible using generic garbling schemes for binary circuits by garbling the encoding function for a HMAC over elliptic curves, but this yields unacceptably large garbling tables. In particular, this is not (much) more efficient than simply garbling the SNARK verifier.

In a nutshell, Argo MAC uses ITPG [11] to evaluate the encoding function for a HMAC over elliptic curve groups. Recall ITPG has complexity dependent on the size of the Arithmetic Branching Program (ABP) for the garbled polynomial, which places strict limits on the complexity of the encoding function. To make this work, we use a HMAC scheme over EC groups whose encoding function is a sparse degree-3 polynomial. Argo MAC uses well-understood, existing techniques based on Yao garbling and ITPG, and only relies on conservative security assumptions about symmetric key cryptography.

¹Designated verifier versions like that described in Glock are able to avoid pairings, with other tradeoffs.

First, we accept the bits of the coordinates of the curve point encoded using standard, Yao-based encoding. Then, we use standard garbling techniques to map this encoding to one compatible with ITPG. Since elliptic curves are algebraic groups, their group law can be expressed using low-degree polynomials. We use ITPG to evaluate the degree-3 encoding function on the input point. Finally, the output of ITPG is a secure MAC of an elliptic curve point that respects the group law.

This technique yields over 3 orders of magnitude performance improvement compared to Yao-based approaches, both in terms of garbling table size, as well as garbler and evaluator speed. Intuitively, this speedup comes from performing as much of the garbling operations natively, i.e. in the group where the objects naturally live, as possible. This avoids the arithmetization overhead of converting high-level operations, like field multiplication, into binary circuits. For example, when computing elliptic curve addition formulae, working with MACs over the base field allows addition operations for free, and multiplication operations much more cheaply, than if they were carried out by a binary circuit.

1.4 Related Work

It is challenging to directly compare Argo MAC to existing work by itself. Most work that would be directly comparable focuses on garbling SNARK verifiers, which we defer. If one were to directly replicate the functionality of Argo MAC using other garbling techniques, the overall protocol would typically be less efficient than directly garbling the verifier. However, assuming that it is possible to construct a garbled SNARK verifier whose table size and garbling time are dominated by Argo MAC, we can estimate what a garbled SNARK verifier would look like using Argo MAC.

There are many garbling schemes for generic boolean circuits [7, 12, 23], which can be used to implement SNARK verifiers. In fact, the most mature approach to this in the context of BitVM/Glock style protocols on Bitcoin is based on privacy-free half gate garbling [24]. This approach incurs large arithmetization overhead when converting operations to binary circuits. This increases garbling and evaluation time by many orders of magnitude compared to the cost of Argo MAC.

Some approaches, such as BitGC [15] or those based on AHMACs [16, 17, 18], are able to avoid increasing the garbling table size, even in the case of AHMAC keeping table size dependent only on input and output size and *not* circuit complexity. However, they are still not able to avoid increasing garbling time, which for a SNARK verifier circuit can be very large [25]. These schemes also have concretely large garbling tables and require additional cryptographic assumptions.

There are also approaches to garbling arithmetic circuits [8, 9, 10]. These come in a number of flavors, but generally require additional heavy cryptographic assumptions or decompose integer wires using the Chinese Remainder Theorem. These schemes can be concretely efficient, and have been applied to BitVM style protocols [26]. While implementation is still ongoing, the resulting tables and garbling times will be larger than Argo MAC.

In forthcoming work, we will show how to build a garbling scheme for pairing-based SNARK verifiers that respects this MAC structure. In different forthcoming work, Babylon Labs [27] will use our technique in a different way to build a SNARK verifier based on linear pairing witness encryption [28]. In both cases, the *key enabling technology* that makes these schemes *over 1000× more efficient* than existing approaches is Argo MAC.

2 Preliminaries

We will use $\mathbb{F}$ to denote fields and $\mathbb{G}, \mathbb{H}$ to denote groups. We write the set of group homomorphisms as $\text{Hom}(\mathbb{G}, \mathbb{H})$, the set of functions as $\mathbb{G} \rightarrow \mathbb{H}$, and the set of partial functions as $\mathbb{G} \rightharpoonup \mathbb{H}$. These partial functions can also be interpreted as functions which may return $\perp$. When $\mathbb{H} = \mathbb{G}$ we denote the integer multiplication homomorphisms by integers, i.e. $n \in \mathbb{Z}$ is the function $n : G \mapsto nG$. When $\mathbb{H} = \mathbb{G}^n$ we denote by vectors $v \in \mathbb{Z}^n$ the homomorphisms $v : G \mapsto (v_i G)_{i \in [n]}$. We adopt the same convention for larger endomorphism rings, for example in curves with complex multiplication (CM).

We will use lowercase letters to refer to field elements and uppercase to curve points. We use both the terms “randomized encodings”, as well as “MAC”/“GC”, where they are more natural, recognizing their equivalence. Throughout E denotes an elliptic curve over base field $\mathbb{F}_q$ with characteristic p and subgroup of size r . We write the cofactor of $E/\mathbb{F}_q$ as h , which by construction satisfies $(r, h) = 1$. We do not assume

anything else about E , but in practice it will typically be one of the groups associated with a pairing-friendly curve from a family like BN [29] or BLS [30].

The scheme is parameterized over the security parameter λ , and a replication factor for the iterated MAC κ . This value is always less than λ and is determined by the CM discriminant and the cofactor of the curve. We denote the number of bits in p by $\rho = \lceil \log_2 q \rceil > 2\lambda$.

For a given MAC scheme we denote the MAC of a value x under the key k using square brackets, i.e. $[x]_k$. When the key is clear from context, we drop the key subscript. We adopt the same notation for garbling schemes. For a given garbling scheme we denote the garbling of a function f under the garbling key k also as $[f]_k$.

2.1 Elliptic Curves

We briefly review the properties of elliptic curves relevant to Argo MAC, and defer a more complete treatment to [31]. An elliptic curve is an algebraic group over a field. We will work with curves in Weierstrass form and make use of their algebraic group laws in a non-blackbox way. The curve is parameterized by two field elements $A, B \in \mathbb{F}_q$ and consists of all the solutions to the equation

$$E/\mathbb{F}_q : y^2 = x^3 + Axz^4 + Bz^6.$$

This is the Jacobian form of the curve, and points are defined to be equivalent if there exists a non-zero r such that $(x : y : z) \sim (r^2x : r^3y : rz)$. The identity element in this form is $\mathcal{O} = (1 : 1 : 0)$. We say a point is “affine” if $z = 1$. For a point P we write its coordinates as $P = (x(P) : y(P) : z(P))$. The inverse of a point has inverted y -coordinate, i.e. $y(-P) = -y(P)$.

The curve group law can be derived using the divisor class group of the curve. There are many different formulae for computing the sum of two points, but we will use the following. These formulae are not complete and are not well-defined if $P, Q = \mathcal{O}$ or $P = \pm Q$. We also assume $z(P) = z(Q) = 1$, and the full formulae can be obtained by substituting $x \mapsto x/z^2$ and $y \mapsto y/z^3$, scaling by $z(P)z(Q)$ subject to Jacobian equivalence.

$$\begin{aligned} x(P + Q) &= x(P)^2x(Q) + x(P)x(Q)^2 - 2y(P)y(Q) + 2B, \\ y(P + Q) &= 3x(P)^2y(Q)x(Q) - 3y(P)x(P)x(Q)^2 + y(P)^2y(Q), \\ &\quad - y(P)y(Q)^2 - 3y(P)B + 3y(Q)B \\ z(P + Q) &= x(P) - x(Q). \end{aligned}$$

Notice these formulae are biquadratic in $(x(P), y(P))$ and $(x(Q), y(Q))$. This allows us to define quadratic equations that add a fixed point to their input.

2.1.1 Complex Multiplication

Since elliptic curves are abelian groups, they admit multiplication by integers. In general, this is the full endomorphism ring of the curve. However, in some cases curves can have larger endomorphism rings which include multiplication by imaginary quadratic integers.

For our purposes, the full theory of complex multiplication is not necessary. It is sufficient to observe that in two cases, there are non-trivial *linear* endomorphisms of a curve. In the first, when $A = 0$ and $\mathbb{F}_q$ has a third root of unity ω , the mapping $P \mapsto (\omega x(P) : y(P) : z(P))$ is a point on the curve. In the second, when $B = 0$ and $\mathbb{F}_q$ has a fourth root of unity i , the mapping $P \mapsto (-x(P) : iy(P) : z(P))$ is a point on the curve. Remarkably, these mappings respect the group law.

2.2 Message Authentication Codes

A MAC is a randomized encoding of some message m under randomness consisting of a key k . A MAC Scheme is defined over a message space $\mathcal{M}$, a key space $\mathcal{K}$, and a MAC space $\mathcal{C}$. It has a function $\text{KeyGen}(1^\lambda) \in \mathcal{K}$, a function $\text{En} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$, and a partial decoding function $\text{De} : \mathcal{K} \times \mathcal{C} \rightharpoonup \mathcal{M}$. For all $\text{key} \in \mathcal{K}$, De is the inverse of En on its image:

$$\begin{aligned} \text{En} : \mathcal{K} \times \mathcal{M} &\rightarrow \mathcal{C}, \\ \text{De} : \mathcal{K} \times \mathcal{C} &\rightharpoonup \mathcal{M}, \\ \text{De}(\text{key}, \text{En}(\text{key}, m)) &= m. \end{aligned}$$

Therefore, MACs must be unique per message. A MAC has t -unforgeability if: given a fixed key $\text{key} = \text{KeyGen}(1^\lambda)$, on input t valid MACs, all PPT adversaries $\mathcal{A}$ have at most negligible advantage in λ of constructing a pair $(m, c) \in \mathcal{M} \times \mathcal{C}$ such that $c = \text{En}(\text{key}, m)$. We say a MAC is hiding if no PPT adversary can distinguish (m_0, c) and (m_1, c) .

2.2.1 Homomorphic MACs

A homomorphic MAC (HMAC) scheme with security parameter λ is parameterized over a pair of groups $\mathbb{G}, \mathbb{H}$ and two subsets $\Phi \subseteq \text{Hom}(\mathbb{G}, \mathbb{H}), K \subseteq \mathbb{H}$ of size at least 2^λ . The message space $\mathcal{M} = \mathbb{G}$, the key space $\mathcal{K} = \Phi \times K$, and the output space $\mathcal{C} = \mathbb{H}$. Key generation simply samples uniformly random ϕ and k . We define encoding and decoding to be:

$$\begin{aligned}\text{KeyGen}(1^\lambda) &:= (\phi, k) \leftarrow \mathcal{K}, \\ \text{En}((\phi, k), m) &:= \phi(m) + k, \\ \text{De}((\phi, k), c) &:= \phi^{-1}(c - k).\end{aligned}$$

For $\mathbb{G} = \mathbb{H} = \mathbb{F}_{2^\lambda}^+$ and natural homomorphisms, this yields the free XOR MAC of free XOR garbling. Similarly, using $\mathbb{G} = \mathbb{H} = \mathbb{F}^+$ we find a MAC that appears implicitly in the randomized encodings of ITPG. We call these MACs homomorphic (indeed they are bilinear) because of the following group homomorphisms:

$$\begin{aligned}\phi &\mapsto \phi(m) + k \in \text{Hom}(\text{Hom}(\mathbb{G}, \mathbb{H}), \mathbb{H}), \\ (m, k) &\mapsto \phi(m) + k \in \text{Hom}(\mathbb{G} \times \mathbb{H}, \mathbb{H}).\end{aligned}$$

In privacy-free schemes, it is convenient to include the message itself in the MAC, which we do implicitly. Except for the key information passed to the ITPG of the encoding function, all our schemes are privacy-free, and we assume all our MACs also carry the message with them.

2.2.2 Iterated MACs

The simplest variant of homomorphic MACs simply lets $\Phi = \text{Hom}(\mathbb{G}, \mathbb{H})$. For many groups, e.g. of order greater than 2^λ , this yields a secure scheme. However, in some cases it is convenient to use a smaller set of homomorphisms. In Argo MAC, this is necessary because we need a set of group homomorphisms that can be evaluated by low-degree polynomials.

Using a common trick, found for example in [16], we can decompose a high-entropy ϕ into a vector of low-entropy ϕ_i 's. Let $\mathbb{H} = \mathbb{H}'^\kappa$ and using $\Phi = \{\phi : \phi_i \in S\}$ where $S \subset \text{Hom}(\mathbb{G}, \mathbb{H}')$ is a small subset of size d . If we have $d^\kappa \geq 2^\lambda$ then $|\Phi| \geq 2^\lambda$ then the set of endomorphisms is still large enough. Generically, given multiple MACs with security levels λ_i over the same value with independent keys, we can concatenate the MACs to construct a MAC with security level $\sum_i \lambda_i$.

2.3 Garbled Circuits

We use substantially the same syntax as [6], where MACs are randomized encodings of values, and garbled circuits are randomized encodings of functions. We define a garbling scheme with respect to a space of functions $\mathcal{F} \subseteq \mathcal{X} \rightarrow \mathcal{Y}$ and two MAC schemes In, Out whose message spaces are $\mathcal{X}$ and $\mathcal{Y}$ respectively.

A garbling scheme is parameterized over a key space $\mathcal{K}$, which includes input and output MAC keys, and a space $\mathcal{G}$ of garblings. It has three functions:

- **KeyGen**. This may either be parameterized over the input and output MAC schemes, or generate the input and output MAC keys.
- **Gb** : $\mathcal{K} \times \mathcal{F} \rightarrow \mathcal{G}$. This accepts a function and a garbling key and returns an element $\text{GC} \in \mathcal{G}$.
- **Ev** : $\mathcal{G} \times \mathcal{C}_{\text{In}} \rightarrow \mathcal{C}_{\text{Out}}$. This accepts a garbling and an input MAC and returns an output MAC of the evaluation of F on that input: $\text{Ev}(\text{Gb}(K, F), [x]_{\text{In}}) = [y]_{\text{Out}}$.

When the input and output MAC schemes of two garbling schemes are the same, we can compose these schemes if they use the same keys. However, the security of the resulting scheme is not immediate. In a privacy-free scheme, all the MACs also include the message, which is the case for our garbling schemes.

When $\mathcal{F} = \{\text{id}\}$ we call the scheme an “adaptor garbled circuit (scheme)”. These allow switching the key while keeping the message the same, which can be useful when moving between different group representations, and in conjunction with the homomorphic properties of HMACs. This is closely related in free XOR MACs with the notion of half gates, and to the construction of ITPG.

A garbling scheme has correctness if **Ev** always returns a valid MAC of y on a valid input MAC. A garbling scheme has authenticity if no PPT adversary can forge an output MAC for a value $y' \neq F(x)$ given a garbling GC and a MAC of x under the input key. A scheme has privacy if no PPT adversary can, for a garbling of F_0 GC, distinguish $(\text{GC}, [x], F_0)$ and $(\text{GC}, [x], F_1)$ where $F_0(x) = F_1(x)$. We prove these properties by simulation arguments, i.e. by showing that there exists a simulator that can produce (at least) two views of the evaluator without knowledge of the garbling key, and that distinguishing these distributions is hard.

2.3.1 Randomized Encodings

The notion of randomized encodings is closely to garbling schemes. It transforms a function f of inputs x and z , using some randomness r , into a randomized encoding $\hat{f}(x, z; r)$. The scheme includes a reconstruction function **Rec** which allows the Evaluator to compute $f(x, z)$ from this encoding and x .

$$\text{Rec}(\hat{f}(x, z; r), x) = f(x, z).$$

Given the randomized encoding and x , the garbler should not learn any information about z apart from $f(x, z)$. Informally, any adversary should not be able to distinguish encodings $\hat{f}(x, z; r)$ and $\hat{f}(x, z'; r')$ where $f(x, z) = f(x, z')$. We call x the “public” input and z the “private” input.

There is a natural relationship with garbling schemes here. Suppose we split the encoding into offline and online parts:

$$\hat{f}(x, z; r) = (\hat{f}_{\text{off}}(r), \hat{f}_{\text{on}}(x, z; r)).$$

The first part looks like a garbling table, the second part like the input encoding, and the reconstruction function like **Ev**. If the function f returns a MAC, then this randomized encoding is equivalent to a (partial) garbling scheme.

In the language of [11], it is partial because x is public to the evaluator. In the case of randomized encodings, purely privacy-free schemes are not interesting since there is only one possible z . However, if z only contains key information about an output MAC, then we recover something like a privacy-free garbling scheme.

2.4 Information Theoretic Partial Garbling

The ITPG protocol [11] is the core component of Argo, so we review it here. The presentation in the paper is general, and in our case we need only a simplification of the protocol for secret evaluation of bivariate, quadratic polynomials. Consider the general bivariate, quadratic polynomial

$$F(z; x, y) = z_0 + z_1x + z_2y + z_3x^2 + z_4xy + z_5y^2.$$

We would like to construct a randomized encoding of this polynomial and reconstruction algorithm which given the encoding and (x, y) can compute $F(x, y)$ without revealing any information about z . To do this, we implicitly construct an ABP with edges for the 5 multiplications by x or y in F . We write the encoding as $e = \hat{F}(r; z, x, y)$.

We begin by defining the affine encoding of the z input as $e_i = z_i + r_i$ for $i \in [0, 5]$. Our reconstruction algorithm will multiply each of these values by the corresponding X, Y monomial, leaving $F(z; x, y)$ plus a polynomial in X, Y, r .

$$F(e_{0:5}; x, y) = F(z; x, y) + F(r_{0:5}; x, y).$$

Now, we will start adding encodings of X and Y to cancel this randomness. For example, adding $e_6 := -r_3x + r_6$ allows the reconstruction algorithm to cancel the x^2 term, $e_7 := -r_4y + r_7$ the xy term, and $e_8 := -r_5y + r_8$ the y^2 term:

$$F(e_{0:5}; x, y) - e_6x - e_7y - e_8y = F(z; x, y) + r_0 + (r_1 + r_6)x + (r_2 + r_7 + r_8)y.$$

Adding two more encodings $e_9 := -(r_1 + r_6 + r_7)x + r_9$ and $e_{10} := -(r_2 + r_8)y - (r_0 + r_9)$ allows us to cancel the remaining terms and define the reconstruction function

$$\begin{aligned}\text{Rec}(e, x, y) &:= F(e_{0:5}; x, y) - e_6x - e_7y - e_8y - e_9 - e_{10} \\ &= F(z; x, y).\end{aligned}$$

It is easy to see, since each term except for the last has a new random value, that the vector e is uniformly randomly distributed among vectors such that reconstruction succeeds. In particular, it is independent of z . For polynomials with fewer non-zero coefficients, like those that occur in our protocol, it is possible to eliminate some of the terms in this reconstruction procedure, like e_5 and e_8 if the polynomial does not have a y -coordinate.

We can interpret this as a garbling scheme and as an input MAC scheme. The input MAC scheme takes the message (x, y) and outputs the encodings $(e_6, \dots, e_{10})$ and the garble function returns $(e_0, \dots, e_5)$. These are the offline and online parts of the encoding respectively.

2.4.1 Non-Linear Reconstruction

For our application, we can apply a simplification of this entirely linear reconstruction algorithm since the private inputs are all fixed in advance. We can eliminate the MACs $e_0, \dots, e_5$ and incorporate the z values into the encoding keys:

$$\begin{aligned}e_5 &= z_5y + r_5 & e_4 &= z_4y + r_4 & e_3 &= z_3x + r_3 \\ e_2 &= (z_2 - r_5)y + r_2 & e_1 &= (z_1 - r_3 - r_4)x + r_1 & e_0 &= z_0 - r_1 - r_2\end{aligned}$$

This allows for the simpler reconstruction procedure

$$\text{Rec}(e, x, y) = e_1 + e_2 + e_3x + e_4x + e_5y.$$

Again, we see that e is uniformly random among vectors that reconstruct the correct value, therefore it is independent of z . Disappointingly, this does not affect the size of the garbling table very much, since the number of elements in the encoding that depend on x and y is the same. In this variant, the input MAC scheme simply returns $(e_1, \dots, e_5)$ and the garble function returns the empty string.

2.5 Half Gates

We adapt the half gate technique [23] to reduce the size of adaptor tables for bits. The simplest construction of an adaptor table using the Yao style technique is constructed from a symmetric encryption algorithm $\mathcal{E}$. Given an input MAC scheme and an output MAC scheme, the garbling table simply stores encryptions of the outputs under the input MACs as keys

$$\text{GC} = (\mathcal{E}([0]_{\text{In}}, [0]_{\text{Out}}), \mathcal{E}([1]_{\text{In}}, [1]_{\text{Out}})).$$

The half gate technique modifies the output MAC by replacing one of these ciphertexts with a hash. We define $[0]_{\text{Out}} = \mathcal{H}([0]_{\text{In}})$ and just store the encryption of 1, $\text{GC} = (\mathcal{E}([1]_{\text{In}}, [1]_{\text{Out}}))$. In the privacy-free setting, the Evaluator knows the message, and so can choose whether to hash or decrypt

$$\text{Ev}(\text{GC}, [x]_{\text{In}}) = \begin{cases} \mathcal{H}([x]_{\text{In}}) & x = 0 \\ \mathcal{D}([x]_{\text{In}}, \text{GC}_1) & x = 1. \end{cases}$$

Proving the security of this scheme with different input MAC schemes is non-trivial, in particular when used with free XOR [12]. In our application, we can choose the input MAC values to be independently, uniformly random, simplifying the security analysis. We only require that $\mathcal{H}([x]_{\text{In}})$ be suitable for use with ITPG: that is, its output should be indistinguishable from random by any PPT adversary.

3 Argo MAC

The Argo MAC protocol is a garbling scheme for the function that takes as input the bits of two field elements (x, y) , and returns a curve point $P = (x, y) \in \mathbb{G}$ if they are its affine Weierstrass coordinates, but fails otherwise. While both the input and output are encodings of the same curve point, they are different objects living in different groups.

$$f_{\text{argo}} : \mathbb{F}_2^\rho \times \mathbb{F}_2^\rho \rightarrow \mathbb{G}$$

$$f_{\text{argo}}(\text{bits}(x), \text{bits}(y)) = \begin{cases} (x, y) & (x, y) \in \mathbb{G} \\ \perp & \text{otherwise} \end{cases}.$$

Note that the input may be poorly formed: for example, the coordinates may not be a curve point, in which case the function returns $\perp$. We can also perform additional input validation, testing that x and y are valid field elements and that $P \in \mathbb{G}$. As we will see later, this is not strictly necessary.

The input MAC scheme is constructed from any MAC scheme for bits, where each bit is encoded under a different, unrelated key. The output MAC scheme is an HMAC where $\mathbb{G} = E/\mathbb{F}_p$ and $\mathbb{H} = \mathbb{G}^\kappa$. We choose our set Φ such that $\phi \in \Phi$ if $\phi_i(P)$ is a degree 1 endomorphism of E . In general this means $\phi_i(P) = \mathcal{O}$ or $\phi_i(P) = P$, but when E has CM discriminant -3 or -4 this can include more functions, resp. 7 and 5. Let the number of degree 1 endomorphisms be d and let $\kappa > \lambda/\log_2 d$. This ensures $|\Phi| = d^\kappa > 2^\lambda$.

To implement this garbling scheme, we split the function into two parts: the output MAC encode function, and input validation.

3.1 Encode

The HMAC encode function naturally splits as κ many independent functions, each computing $[P]_i = \phi_i(P) + K_i$. Let $\mathbb{K} = \mathbb{F}_p(E)$ be the function field of the curve. We drop the i subscript for now and consider a single component of the MAC. Recall that for Weierstrass curves, this function can be written using three quadratic polynomials $f_x, f_y, f_z \in \mathbb{K}[X, Y]$ such that for all K and any $P \notin \{K, \mathcal{O}\}$,

$$\phi(P) + K = \left(\frac{f_x(x(P), y(P))}{f_z(x(P), y(P))^2}, \frac{f_y(x(P), y(P))}{f_z(x(P), y(P))^3} \right).$$

These polynomials return the Jacobian coordinates of the result, so for any $r \in \mathbb{F}_p^\times$ the polynomials $(r^2 f_x, r^3 f_y, r f_z)$ yield the same point up to equivalence. For each of $v \in \{x, y, z\}$ we can decompose f_v into a sum of at most n_v products of coefficients $\vec{c}_{v,j}^{(\phi)} \in \mathbb{K}[r]$ and monomials $\vec{m}_{v,j} \in \mathbb{F}[X, Y]$, where $j \in [n_v]$. Define the generic $F_v(\vec{C}_v; X, Y) \in \mathbb{F}_p[\vec{C}_v, X, Y]$ such that $f_v(X, Y) = F_v(\vec{c}_v^{(\phi)}(r, K); X, Y)$. Concretely

$$\begin{aligned} F_x(\vec{C}_x; X, Y) &= C_{x,0} + C_{x,1}X + C_{x,2}Y + C_{x,3}X^2 \\ F_y(\vec{C}_y; X, Y) &= C_{y,0} + C_{y,1}Y + C_{y,2}X^2 + C_{y,3}XY + C_{y,4}Y^2 \\ F_z(\vec{C}_z; X, Y) &= C_{z,0} + C_{z,1}X. \end{aligned}$$

Because the endomorphisms are always linear, this is the same degree and shape of polynomials as Jacobian point addition.

The key observation is that F_v is linear in the coefficient variables and that all the secret information about the key is encoded into these variables. Therefore, we can apply ITPG to the function F_v to construct a randomized encoding $\hat{F}_v(\vec{c}_v^{(\phi)}(r, K), x(P), y(P))$ that hides the $\vec{c}_v^{(\phi)}(r, K)$ and still allows the evaluator to compute $[P]$. For curves in Weierstrass form, the total number of edges in the ABPs for f_x, f_y, f_z is 9.

Using the same technique as in the original paper, we can lift F_v from polynomials in the coordinates of the curve points to polynomials in the bits of the coordinates as follows:

$$G_v(\vec{C}_v; \vec{X}, \vec{Y}) = F_v\left(\vec{C}_v, \sum_{j=0}^{\rho-1} X_j 2^j, \sum_{j=0}^{\rho-1} Y_j 2^j\right).$$

The scheme then yields a randomized encoding of G_v that consists entirely of affine functions of the variables $\vec{X}$ and $\vec{Y}$, which is equivalently an HMAC where $\mathbb{G} = (\mathbb{F}_p^+)^{2\rho}$ and $\mathbb{H} = (\mathbb{F}_p^+)^{9\rho}$. For a suitably defined MAC key and input $P \in E$ we therefore have

$$\hat{G}_v(\vec{c}_v^{(\phi_i)}(r_i, K_i); \text{bits}(x(P)), \text{bits}(y(P))) = v([P]_i).$$

We can further concatenate all these HMACs across all $i \in [\kappa]$ to construct a single HMAC from $\mathbb{G}$ to $\mathbb{H} = (\mathbb{F}_p^+)^{9\rho\kappa}$. Equivalently, we have 2ρ HMACs, one for each bit of input. The HMACs for the bits of x map to $5\rho\kappa$ field elements and the HMACs for the bits of y to $4\rho\kappa$.

3.1.1 Using Half Gates

To complete the GC, we simply need an adaptor table from the input MAC to this HMAC for the message space $\mathcal{M} = \{0, 1\}$. Using the half-gate technique, this requires one ciphertext per component of $\mathbb{H}$, or $9\rho^2\kappa$ bits. For a curve over a 256-bit field with CM discriminant -3 and security level 128, this comes to about 3.6MB.

This yields a correct protocol because for each encoding of a value, the constant term in the affine function is of the form r_i , that is an independent random value. Thus, we can use $\mathcal{H}$ to generate these values. One term in the reconstruction does not have a constant term of this form, but that element of the encoding does not depend on the input.

3.2 Validation

There are three components to input validation: (1) checking that the coordinates of the curve point are valid field elements, (2) checking that the coordinates form a valid curve point, and (3) checking that the curve point lies in $\mathbb{G}$. All of these need only be done once, not replicated κ times like the encode function, and therefore contribute very little to the overall cost of the scheme.

There is an interesting subtlety in (1) and (3) since it is not a priori defined what it means to be an invalid field element or element of the subgroup. We can define both of these as equivalence classes over a larger set, and we could choose to define all elements of the larger set as valid and simply quotient by the equivalence class. Alternatively, and more commonly, we could require that the input be a canonical representative of the equivalence class. In either case, there is no sensible way to quotient an arbitrary pair of field elements to a curve point, so condition (2) can always fail.

3.2.1 Field Element Validity

Normalization. The $\mathbb{F}_p^+$ MACs naturally quotient alternative bit representations for the same field element, so this happens for free. We simply compute a power of 2 linear combination of MACs, which always yields the same result since the MACs are $\mathbb{F}$ linear.

Canonicity. We can use an existing garbling scheme to evaluate a `LessThan` circuit on the input and the field characteristic. This is a very small circuit with one `AND` gate per bit. There are many candidates, but privacy-free half gates with free `XOR` yields the smallest circuit. If the circuit fails we reveal the MAC of $\perp$.

3.2.2 Curve Point Validity

A curve point is valid if and only if it is a solution to the curve equation, which in Weierstrass form is $y^2 = x^3 + Ax + B$. To check curve point validity, we can reuse the ITPG scheme for leaking a secret if a polynomial equation is not satisfied. That is, for a secret s apply the scheme to the polynomial

$$f_{\text{curve}}(X, Y) = (X^3 + AX + B - Y^2)s.$$

After using ITPG, we divide the result by $X^3 + AX + B - Y^2$ if the curve point is non-zero. We can let $s = [\perp]$ or use an adaptor table to reveal the MAC of $\perp$ given s .

3.2.3 Subgroup Membership

Normalization. Let h be the cofactor of the elliptic curve: that is, $\#E/\mathbb{F}_p = rh$. Every curve point decomposes uniquely as $P = R + H$ where $rR = hH = \mathcal{O}$. Let R be the unique representative of the equivalence class E quotiented by the h order subgroup. Let c be the unique integer in $[0, rh - 1]$ such that $c = 0 \bmod h$ and $c = 1 \bmod r$. Multiplying, $cP = R$. Therefore, if K is in $\mathbb{G}$, we can compute a MAC of the normalized point P by multiplying by c ,

$$c[P] = \phi(R) + K.$$

Canonicity. To efficiently test whether $P \in \mathbb{G}$, let $\mathbb{G}' = E[h]$ be the cofactor group not including $\mathcal{O}$. We must choose Φ such that for $Q \in \mathbb{G}'/\mathcal{O}$, $\phi \mapsto \phi(Q)$ is injective. This constrains the possible Φ based on the factors of the cofactor. For example, if $2 \mid h$ then $\Phi = \{0, 1\}^\kappa$ is the only possible choice. If $\gcd(h, 12) = 1$ then we can always use the full CM endomorphism set.

Observe that if $P \notin \mathbb{G}$, then $P = R + H$ for non-zero H , and that $[P]_i = [R]_i + \phi(H)$. Multiplying by r , we find $r[P]_i = \phi_i(rH)$. Since evaluation by ϕ_i is invertible in the cofactor group, except on zero, we can solve the discrete log and compute ϕ if and only if $H \neq \mathcal{O}$. Therefore, if $P \notin \mathbb{G}$ we leak ϕ and if $P \in \mathbb{G}$ we do not leak ϕ . This does not require any modifications to the garbling table. Again, we can use an adaptor table to leak $[\perp]$ given ϕ .

3.3 Incomplete Addition Formulas

Recall that Jacobian point addition formulae are incomplete. When we evaluate $P + P$, the formulae will return the non-curve point value $(0 : 0 : 0)$. They are also incomplete for the point at infinity, but it is not possible to encode this as an affine point. So, if $\phi(P) = K$, simply evaluating the polynomials will fail to return a valid curve point, so the MAC is not well defined. In applications related to BitVM and Glock, the Garbler is able to freely choose the input point. This means a malicious Garbler could maliciously cause the scheme to output an invalid curve point.

To solve this, we augment the Jacobian addition formulae with an additional term $u_i x(2P)$. The value u_i is 1 if $P = \phi^{-1}(K)$ and zero otherwise. We can compute this using an equality circuit between $x(P) = x(\phi^{-1}(K))$ and the top bits of $y(P)$ and $y(\phi^{-1}(K))$. If the points are equal, we learn a secret that allows us to learn an ITPG encoding of $g = 1$. Otherwise, we learn a secret that allows us to learn the encoding of $g = 0$. We can use a simple Yao-style garbling scheme for this step. The updated functions to garble are

$$f'_v(P) = f_v(P) + gv(2P).$$

In many applications the Garbler has freedom to choose which points to MAC. For example, in a SNARK verifier the points are likely distributed uniformly at random, so the probability of a failure occurring is negligible. In that case, it may be sufficient to simply leak a secret directly if a failure occurs, as it can only happen with a malicious Garbler. This does not even require a Yao-style garbling table for equality. We can leak a secret directly by hashing the input labels and computing their XOR. If points are equal, then the resulting value can be used to decrypt a secret. Otherwise, the Evaluator learns nothing.

3.4 Protocol

Argo MAC Output MAC Scheme

1. Let E be an elliptic curve and S the set of d endomorphisms of degree 1
2. Let $\kappa = 2^\lambda / \log_2 d$
3. $\mathbb{G} = E \quad \mathbb{H} = \mathbb{G}^\kappa$
4. $K = \mathbb{H} \quad \Phi = \{\phi \neq 0 : i \in [\kappa], \phi_i \in S\} \subset \text{Hom}(\mathbb{G}, \mathbb{H})$
5. $\text{En}((\phi, K), P) = \phi(P) + K$

6. Note since $\phi \neq 0$ the decode function will always have at least one entry j where $\phi_j \neq 0$

$$\text{De}((\phi, K), [P]) = \begin{cases} \phi_j^{-1}([P]_j - K_j) & \forall i : [P]_i = \phi_i(P) + K_i \\ \perp & \text{otherwise.} \end{cases}$$

Argo MAC Garbling Scheme

1. Let PRG be a pseudorandom generator
2. Let $\mathcal{H}(x) = \text{PRG}(x)$ and $\mathcal{E}(k, m) = m + \text{PRG}(k)$
3. Let LT_x be a garbling scheme for the function $y \mapsto y < x$
4. In the case of extension fields, let LT_x apply component-wise
5. Let EQ_x be a garbling scheme for the function $y \mapsto y = x$
6. Let the garbling key be:
 - a) All the input MAC keys
 - b) All the GC keys for EQ and LT
 - c) (r_i, ϕ_i, K_i) for all $i \in \kappa$.
 - d) The value $[\perp]$.

Garble

1. Construct a GC for LT_q for each input scalar.
2. Construct a GC for EQ for each $x(K_i), \text{bit}_{\rho-1}(y(K_i))$.
3. Adaptor tables from input bits and outputs of EQ_x to $(\mathbb{F}_p^+)^{(9\rho+1)\kappa}$
4. Use the output MAC scheme from the adaptor tables in the ITPG encoding for the encoding function and f_{curve}
5. Use $[\perp]$ as s in f_{curve}
6. The garbled circuit includes
 - a) The tables for EQ and LT
 - b) Adaptor table from LT to $[\perp]$
 - c) Adaptor tables from EQ to the encoding for f_{argo}
 - d) Adaptor tables from input to the encoding for f_{argo}
 - e) Adaptor tables from input to the encoding for f_{curve}

Evaluate

1. Evaluate the LT circuits and if any return 0 use the associated adaptor table to derive $[\perp]$
2. Evaluate the EQ circuits to derive encodings of g_i for $i \in \kappa$.
3. Use the appropriate adaptor tables to derive the input encoding for f_{argo} and f_{curve} .
4. If the input point is not a valid curve point, reconstruct the output of f_{curve} which is $[\perp]$.
5. Reconstruct all the $[P]_i$.
6. If $P \notin \mathbb{G}$ clear the $\mathbb{G}$ part and find ϕ . Use the appropriate adaptor table to derive $[\perp]$.
7. Otherwise, return $[P]$.

3.5 Security

Essentially all of the components of Argo MAC already have security proofs. In fact, the ITPG paper has a verifiable computation protocol very similar to ours. The only non-trivial things to prove are that randomizing the Jacobian equivalence class is secure, and that using ITPG with half gates is secure.

We would like to use the randomness generated by a hash function $\mathcal{H}$ in the half gates as the randomness for ITPG. The security of this follows almost immediately by assumption that the output of $\mathcal{H}$ is indistinguishable from pseudorandom if we do not know the input to the function. For practical efficiency, we use AES to instantiate this function.

In practice, we also assume that the garbler has given a commitment to the input labels to the Evaluator. This is necessary for BitVM-style protocols on Bitcoin and can be instantiated with a collision-resistant, one-way function. However, this is outside the scope of this paper.

3.5.1 Jacobian Coordinates Proof

While the ITPG scheme ensures that an adversary cannot learn any information about z from the encoding, it is important to show that the function itself does not reveal any information about the MAC key. As a strawman, observe that if we do not randomize up to Jacobian equivalence, the output can leak information about the key. Suppose $\phi = 0$, in which case if $f_z(P) = 1$. Now suppose $\phi \neq 0$, in which case $f_z(P) = x(P) - x(K) \neq 0$ with high probability. Therefore, naively applying the ITPG protocol to this function *would* leak ϕ with high probability.

To show security, we show that for any non-zero input P , output triple $(x : y : z)$, and any ϕ , there exists a pair (r, K) such that

$$x = r^2 f_x(P) \quad y = r^3 f_y(P) \quad z = r f_z(P).$$

Take the point $R = (x : y : z)$. Now, let $K = R - \phi(P)$ and compute the non-randomized coefficients using (ϕ, K)

$$x' = f_x(P) \quad y' = f_y(P) \quad z' = f_z(P).$$

Because of the group law, we know that $(x' : y' : z') \sim (x, y, z)$ and therefore there exists some $r \neq 0$ such that $x = r^2 x', y = r^3 y', z = r z'$. Therefore, there exists exactly one (r, K) for each $((x : y : z), P, \phi)$. Symmetrically, for uniformly random (r, ϕ, K) the value $[P]$ is uniformly randomly distributed among $(x : y : z)$ satisfying the curve equation. Equivalently, it is a random element of a random equivalence class and therefore $[P]$ is independent of the key.

3.5.2 Overall Proof

We prove security of the overall protocol using a simulation-based argument for authenticity, in the syntax of [6]. This is the only property we care about because the scheme is privacy-free. The simulator will take as input a curve point P and a randomness oracle RNG, and it will output an input encoding and a garbled circuit. The simulator will execute the garbling procedure as specified, but will generate the unopened branch using RNG. If $\text{RNG} = \text{PRG}$, then the resulting GC will be from the same distribution as valid garbled circuits. If $\text{RNG} \neq \text{PRG}$ is a true random number generator, then the resulting distribution will be uniformly random among all circuits. This is because the adaptor tables consist only of $\text{PRG}(\ell_b) + \text{RNG}(\ell_{1-b})$.

Suppose there exists a PPT adversary that can distinguish these two cases with non-negligible advantage. Then, we can build a distinguisher for PRG from real randomness using the adversary. This distinguisher will succeed with advantage equal to that of the adversary, which is non-negligible. This is impossible by assumption on the PRG and therefore $\mathcal{A}$ cannot exist. Therefore, the distribution of real $([x], \text{GC})$ is indistinguishable from uniformly random and is independent of the garbling key.

We apply the same argument to show that all adaptor tables generated during garbling are independent of the garbling key. Note that the tables we use for gluing the **LessThan** circuit and secret revelation are *half-gate* tables, so the proof is simpler. To argue for the overall security of the scheme, we only need to invoke the authenticity property of the **LessThan** circuit used for input validation. If this scheme allows setting the output MAC key during garbling, we can sample it ourselves and we are done. If it does not, we can use the output MAC key chosen by the **LessThan** scheme and let $[\perp]_{\text{argo}} = [0]_{\text{lt}}$.

4 Performance

We implemented Argo MAC in under 2500 lines of code, using finite field and elliptic curve primitives from the popular `arkworks` [32] Rust cryptography library. In Table 1, we report garbling times, evaluation times, and table sizes for Argo MAC instantiated over both $\mathbb{G}_1$ and $\mathbb{G}_2$ groups of the BN254 curve. κ_A and κ_B are defined as $\kappa_A = \rho / \log_2(d)$, $\kappa_B = \lambda / \log_2(d)$. Our implementation uses $\lambda = 128$, $\rho = 256$, $d = 7$.

Group	κ	Garbling (ms)	Evaluation (ms)	Table size (MB)
$\mathbb{G}_1$	κ_A	45	22	7.1
$\mathbb{G}_2$	κ_B	72	37	13.9
$\mathbb{G}_1$	κ_B	23	11	3.6

Table 1: Garbling times, evaluation times, and table sizes for $\mathbb{G}_1$ and $\mathbb{G}_2$ elements. These benchmarks were run single-threaded, on a machine with an Apple M4 chip and 24GB RAM.

Since Argo MAC is the dominant cost in the table size of the full Groth16 verifier garbling scheme, these benchmarks allow us to make a preliminary estimate of the full scheme’s table size. Recall that a Groth16 proof consists of $(A, B, C, x) \in (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_1, \mathbb{F}_q)$. Due to the structure of the pairing gadget in the full scheme, we will use κ_A for A , and κ_B for B, C , giving us a Groth16 verifier garbling table of size roughly 25MB.

Acknowledgements

Thanks to Alpen Labs for supporting the first author’s research on precursors to this scheme. Thanks to the Babylon Labs team for conversations about the scheme.

References

- [1] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [2] Robin Linus. *BitVM: Compute Anything on Bitcoin*. 2023. URL: <https://bitvm.org/bitvm.pdf>.
- [3] Lukas Aumayr et al. *BitVM: Quasi-Turing Complete Computation on Bitcoin*. Cryptology ePrint Archive, Report 2024/1995. 2024. URL: <https://eprint.iacr.org/2024/1995>.
- [4] Robin Linus et al. *Bridging Bitcoin to Second Layers via BitVM2*. Cryptology ePrint Archive, Report 2025/1158. 2025. URL: <https://eprint.iacr.org/2025/1158>.
- [5] Liam Eagen. *Glock: Garbled Locks for Bitcoin*. Cryptology ePrint Archive, Report 2025/1485. 2025. URL: <https://eprint.iacr.org/2025/1485>.
- [6] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. *Foundations of Garbled Circuits*. Cryptology ePrint Archive, Report 2012/265. 2012. URL: <https://eprint.iacr.org/2012/265>.
- [7] Andrew C. Yao. “How to Generate and Exchange Secrets”. In: *27th Annual Symposium on Foundations of Computer Science (FOCS)*. DOI: 10.1109/SFCS.1986.25.
- [8] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. *How to Garble Arithmetic Circuits*. Cryptology ePrint Archive, Report 2012/255. 2012. URL: <https://eprint.iacr.org/2012/255>.
- [9] Marshall Ball et al. *New Ways to Garble Arithmetic Circuits*. Cryptology ePrint Archive, Report 2023/501. 2023. URL: <https://eprint.iacr.org/2023/501>.
- [10] David Heath. *Efficient Arithmetic in Garbled Circuits*. Cryptology ePrint Archive, Report 2024/139. 2024. URL: <https://eprint.iacr.org/2024/139>.
- [11] Yuval Ishai and Hoeteck Wee. *Partial Garbling Schemes and Their Applications*. Cryptology ePrint Archive, Report 2014/995. 2014. URL: <https://eprint.iacr.org/2014/995>.
- [12] Vladimir Kolesnikov and Thomas Schneider. “Improved Garbled Circuit: Free XOR Gates and Applications”. In: *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008*. DOI: 10.1007/978-3-540-70583-3_40.

- [13] Jae Hyun Ahn et al. *Computing on Authenticated Data*. Cryptology ePrint Archive, Report 2011/096. 2011. URL: <https://eprint.iacr.org/2011/096>.
- [14] Rosario Gennaro and Daniel Wichs. *Fully Homomorphic Message Authenticators*. Cryptology ePrint Archive, Report 2012/290. 2012. URL: <https://eprint.iacr.org/2012/290>.
- [15] Hanlin Liu et al. *BitGC: Garbled Circuits with 1 Bit per Gate*. Cryptology ePrint Archive, Report 2024/1988. 2024. URL: <https://eprint.iacr.org/2024/1988>.
- [16] Yuval Ishai, Hanjun Li, and Huijia Lin. *Succinct Homomorphic MACs from Groups and Applications*. Cryptology ePrint Archive, Report 2024/2073. 2024. URL: <https://eprint.iacr.org/2024/2073>.
- [17] Yuval Ishai, Hanjun Li, and Huijia Lin. *A Unified Framework for Succinct Garbling from Homomorphic Secret Sharing*. Cryptology ePrint Archive, Report 2025/442. 2025. URL: <https://eprint.iacr.org/2025/442>.
- [18] Hanjun Li, Huijia Lin, and George Lu. *Succinct Garbled Circuits with Low-Depth Garbling Algorithms*. Cryptology ePrint Archive, Report 2025/2308. 2025. URL: <https://eprint.iacr.org/2025/2308>.
- [19] Robin Linus. *Optimizing On-Chain Costs for Publishing Proofs in BitVM-Style Bridges*. 2025. URL: <https://gist.github.com/RobinLinus/0fc7405ad7485c35465efb7996a7b014>.
- [20] Harry A. Kalodner et al. “Arbitrum: Scalable, private smart contracts”. In: *USENIX Security 2018*. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner>.
- [21] Jens Groth. *On the Size of Pairing-based Non-interactive Arguments*. Cryptology ePrint Archive, Report 2016/260. 2016. URL: <https://eprint.iacr.org/2016/260>.
- [22] Michel Dellepère, Pratyush Mishra, and Alireza Shirzad. *Garuda and Pari: Faster and Smaller SNARKs via Equi-efficient Polynomial Commitments*. Cryptology ePrint Archive, Report 2024/1245. 2024. URL: <https://eprint.iacr.org/2024/1245>.
- [23] Samee Zahur, Mike Rosulek, and David Evans. *Two Halves Make a Whole: Reducing Data Transfer in Garbled Circuits using Half Gates*. Cryptology ePrint Archive, Report 2014/756. 2014. URL: <https://eprint.iacr.org/2014/756>.
- [24] BitVM Alliance. *Garbled SNARK Verifier (BitVM)*. 2025. URL: <https://github.com/BitVM/garbled-snark-verifier>.
- [25] Andrew Ong. *ZeroGC: A Garbled Circuit Scheme for BitVM3 with Zero Ciphertext per Gate*. 2025. URL: <https://hackmd.io/@bitlayer/BJEieeTSeg>.
- [26] Ariel Futoransky et al. *OHMG: One hot modular garbling*. Cryptology ePrint Archive, Paper 2025/2338. 2025. URL: <https://eprint.iacr.org/2025/2338>.
- [27] Babylon Labs. URL: <https://babylonlabs.io>.
- [28] Sanjam Garg et al. *A Framework for Witness Encryption from Linearly Verifiable SNARKs and Applications*. Cryptology ePrint Archive, Report 2025/1364. 2025. URL: <https://eprint.iacr.org/2025/1364>.
- [29] Paulo S. L. M. Barreto and Michael Naehrig. *Pairing-Friendly Elliptic Curves of Prime Order*. Cryptology ePrint Archive, Report 2005/133. 2005. URL: <https://eprint.iacr.org/2005/133>.
- [30] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. *Constructing Elliptic Curves with Prescribed Embedding Degrees*. Cryptology ePrint Archive, Report 2002/088. 2002. URL: <https://eprint.iacr.org/2002/088>.
- [31] Lawrence C. Washington. *Elliptic Curves: Number Theory and Cryptography*. 2nd. Chapman and Hall/CRC, 2008.
- [32] arkworks contributors. *arkworks zkSNARK ecosystem*. URL: <https://github.com/arkworks-rs/>.